

ECE 4502/6502 & CS 6501: Machine Learning on Graphs

Lecture 10. Network Embedding I

Instructor: Jundong Li
Spring 2025, University of Virginia

Administrivia

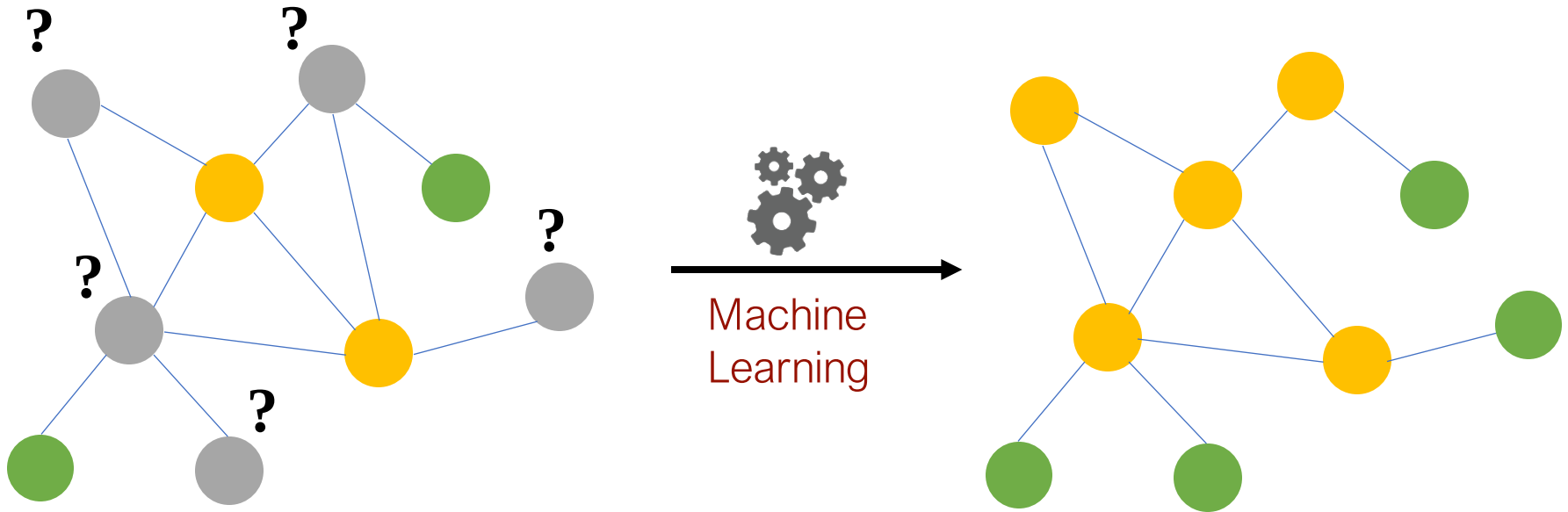
- Homework 2 released today
- Deadline: **March 26th, Sunday**

Machine Learning with Networks

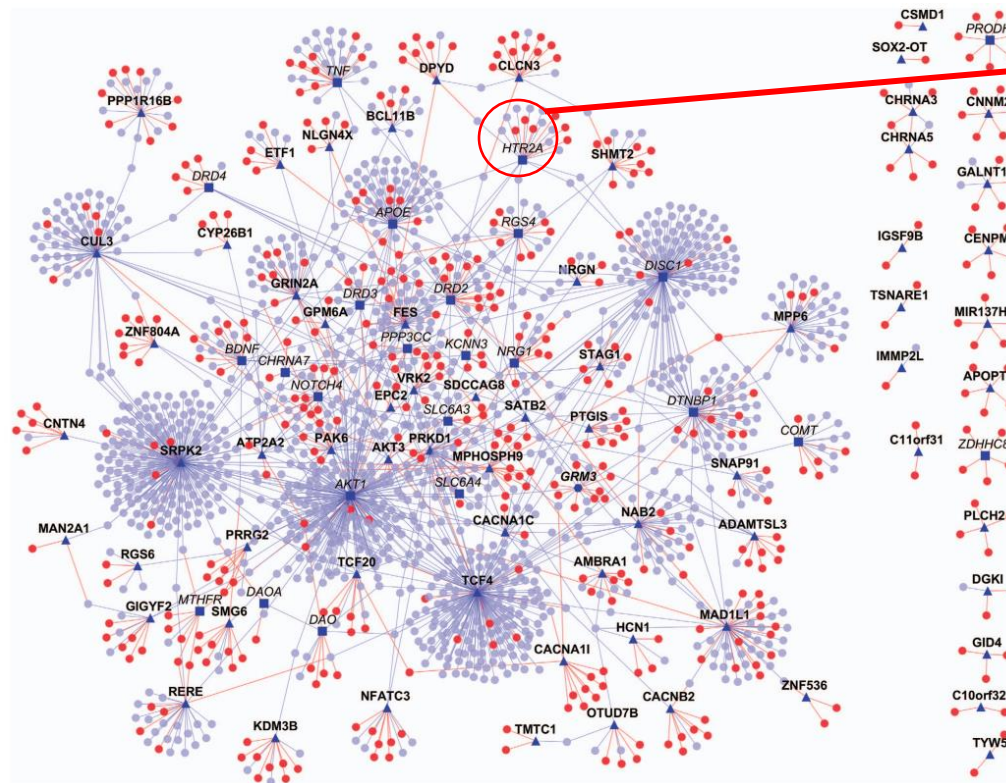
Classical ML tasks in networks:

- Node classification
 - Predict the class label of a given node
- Link prediction
 - Predict whether two nodes are linked
- Community detection
 - Identify densely linked clusters of nodes
- Anomaly detection
 - Find suspicious patterns of nodes on graphs

Example: Node Classification



Protein Function Prediction

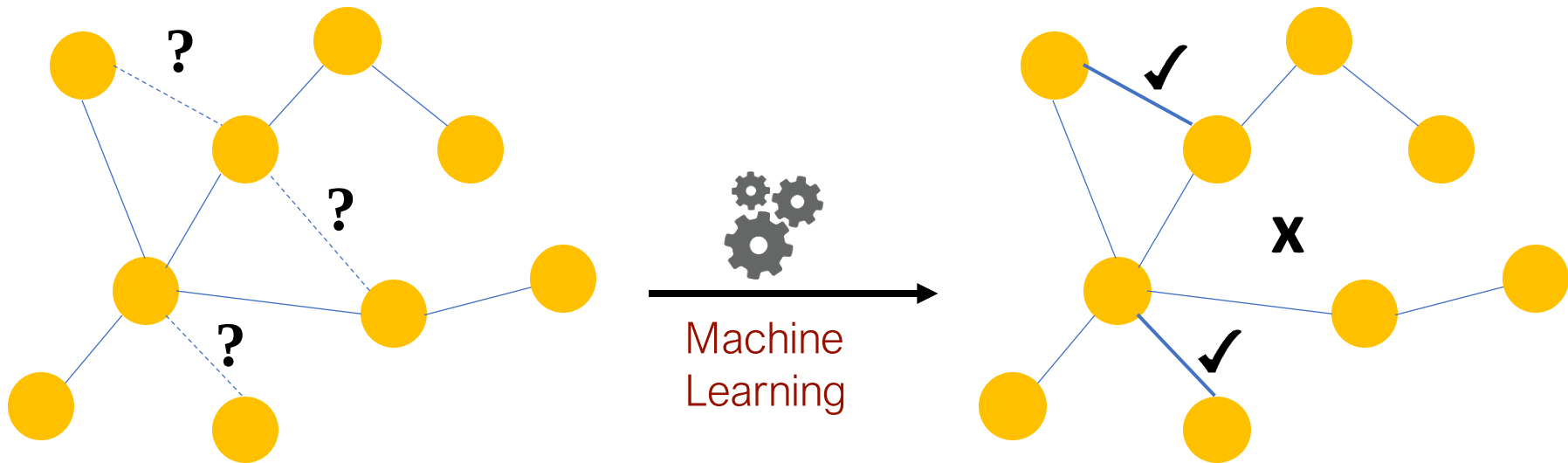


SHMT2:

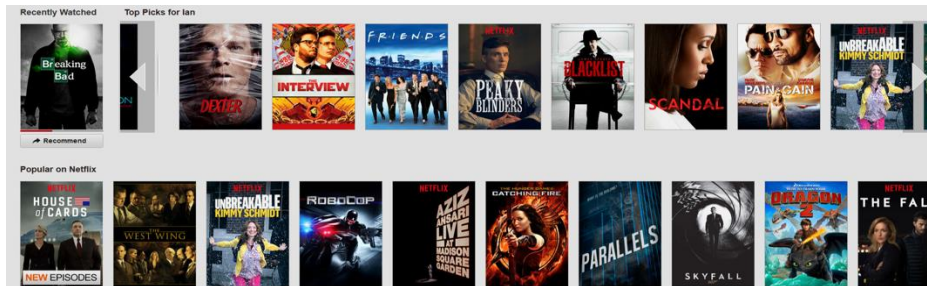
{YOR115C, YDR108W,
YDR472W, ...}

**Predicting the missing labels of nodes
on the graph – node classification**

Example: Link Prediction



Recommender Systems



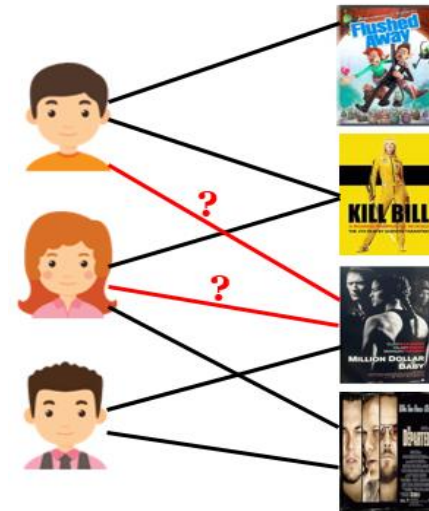
Movies You've Rated

Based on your 745 movie ratings, this is the list of movies you've seen. As you discover movies on the website that you've seen, rate them and they will show up on this list. On this page, you may change the rating for any movie you've seen, and you may remove a movie from this list by clicking the 'Clear Rating' button.

Sort by >

Jump to >

	TITLE	MPAA	GENRE	STAR RATING
Add	12 Angry Men (1957)	UR	Classics	5 stars Clear Rating
Add	The 39 Steps (1935)	UR	Classics	5 stars Clear Rating
Add	An American in Paris (1951)	UR	Classics	5 stars Clear Rating
Add	The Andromeda Strain (1971)	G	Sci-Fi & Fantasy	5 stars Clear Rating
Add	Apollo 13 (1995)	PG	Drama	5 stars Clear Rating
Add	The Battle of Algiers (1965) La Battaglia di Algeri	UR	Foreign	5 stars Clear Rating
Add	Being There (1979)	PG	Drama	5 stars Clear Rating
Add	Big Deal on Madonna Street (1958) I soldi ignoti	UR	Foreign	5 stars Clear Rating
Add	The Birds (1963)	PG-13	Thrillers	5 stars Clear Rating
Add	Blade Runner (1982)	R	Sci-Fi & Fantasy	5 stars Clear Rating



Recommendation is essentially a link prediction problem on bipartite graphs

Traditional Machine Learning

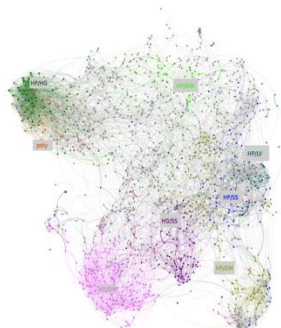
- Traditional machine learning pipeline requires feature engineering **every single time!**



~~Feature Engineering~~

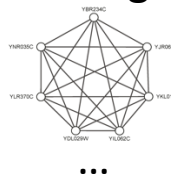
Automatically learn the features

Downstream prediction task



Node degree

Clustering effect

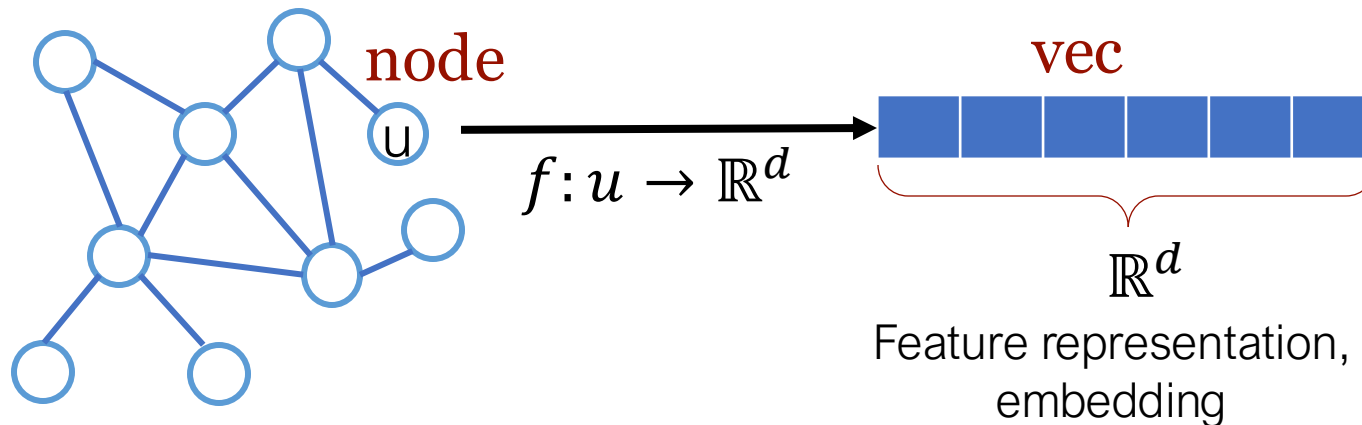


Learning algorithms on graphs

- Node classification
- Community detection
- Link prediction
- Anomaly detection
- ...

Feature Learning in Graphs

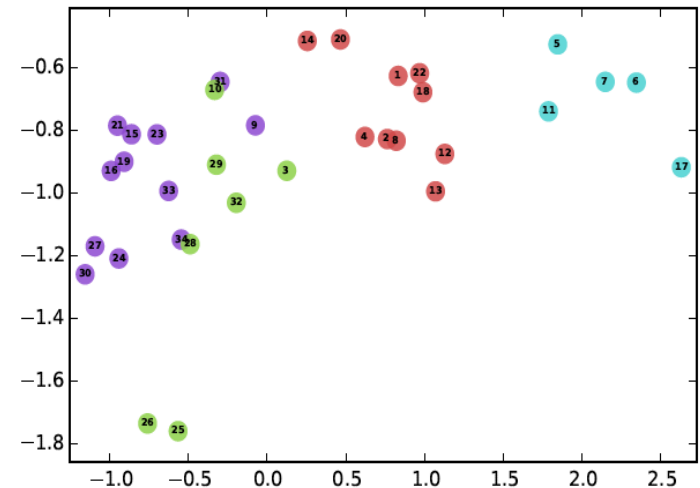
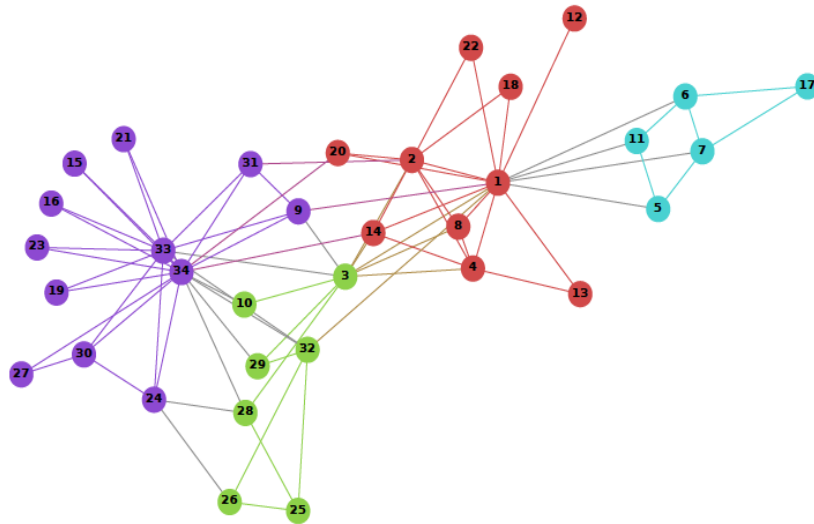
- **Goal:** Efficient task-independent feature learning for machine learning in networks (a.k.a. network/graph embedding)



Example

- Zachary's Karate Club Network:

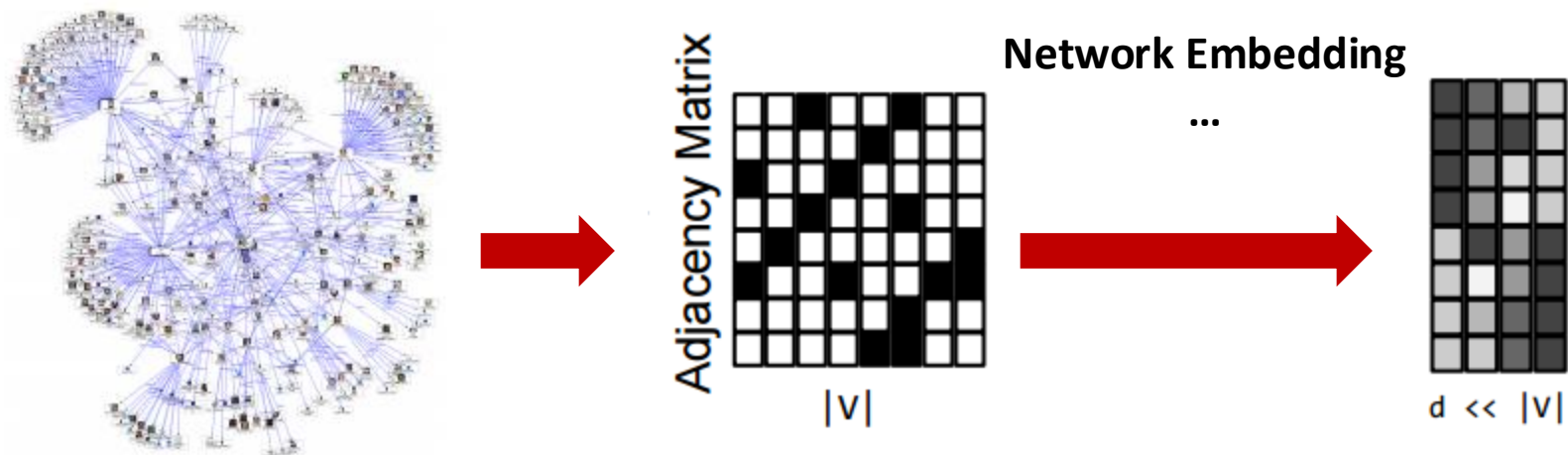
Image from: [Perozzi et al. 2014](#). DeepWalk: Online Learning of Social Representations. *KDD 2014*.



Intuition: Find embedding of nodes to d -dimensions so that “similar” nodes in the graph have embeddings that are close together.

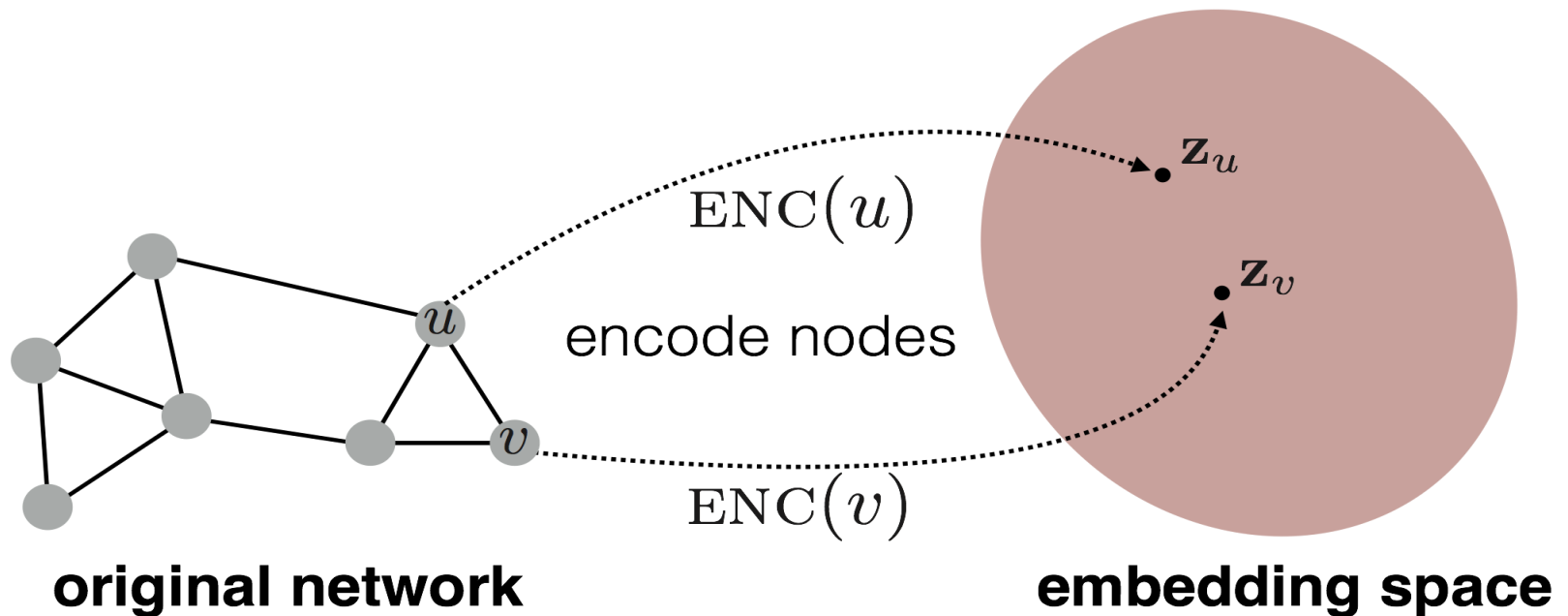
Problem Setup

- Input: graph $G = (V, E)$
 - V is the vertex set (n nodes in total).
 - A is the adjacency matrix (assume binary).
 - **No node features or extra information is used!**
- Output: d-dimensional vector for each node



Embedding Nodes

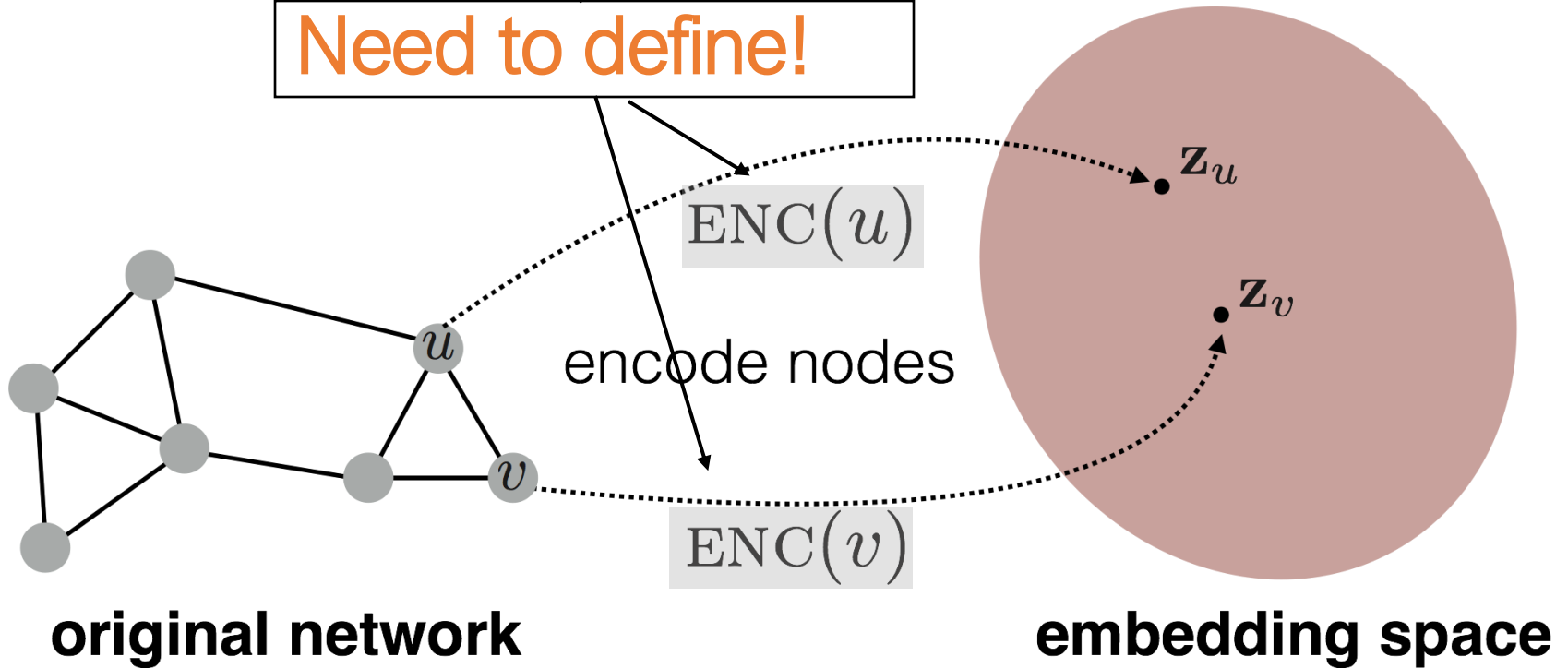
- Goal is to encode nodes so that **similarity in the embedding space (e.g., dot product)** approximates **similarity in the original network**.



Embedding Nodes

Goal: $\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$

Need to define!



Learning Node Embeddings

- 1. Define an encoder** (i.e., a mapping from nodes to embeddings)
- 2. Define a node similarity function** (i.e., a measure of similarity in the original network)
- 3. Optimize the parameters of the encoder so that:**

$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$

Two Key Components

- **Encoder** maps each node to a low-dimensional vector

$$\text{ENC}(v) = \mathbf{z}_v$$

node in the input graph

d-dimensional embedding

- **Similarity function** specifies how relations in vector space map to relations in the original network

$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$

Similarity of u and v in the original network

dot product between node embeddings

“Shallow” Encoding

- Simplest encoding approach: **encoder is just an embedding-lookup**

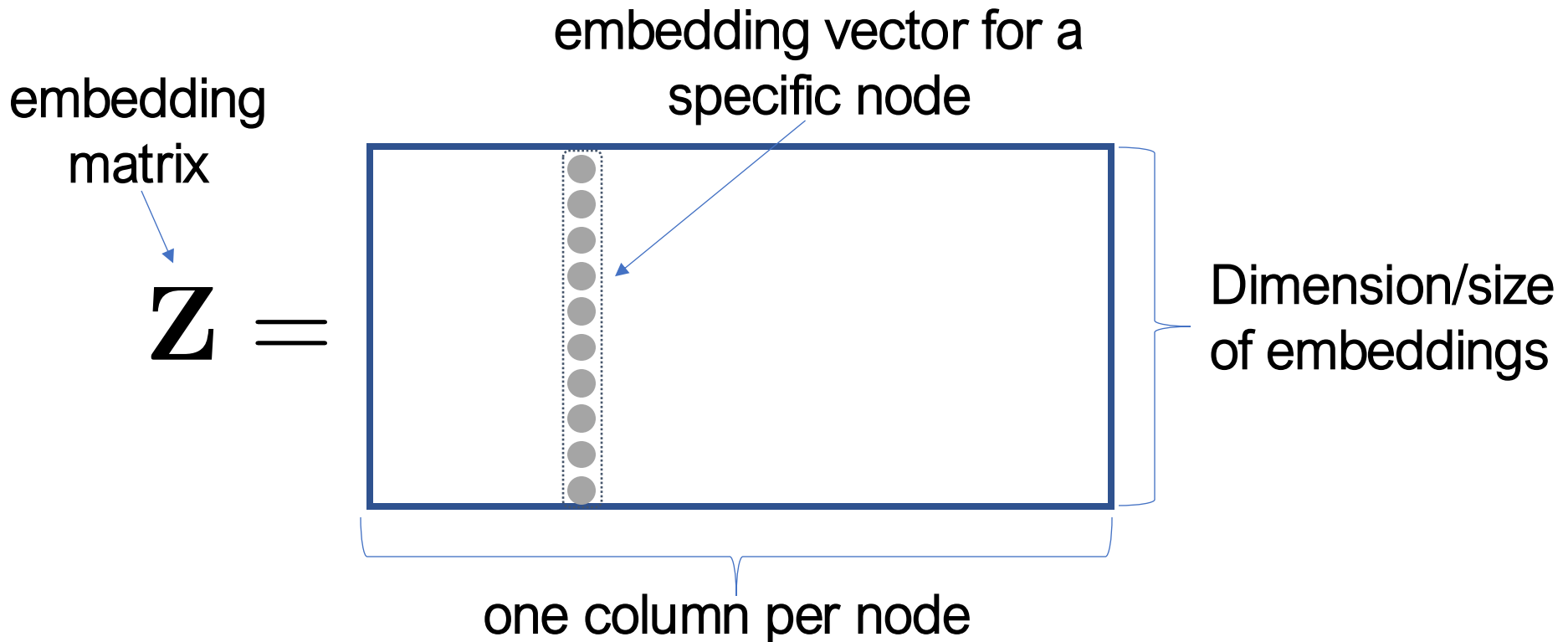
$$\text{ENC}(v) = \mathbf{Z}\mathbf{v}$$

$\mathbf{Z} \in \mathbb{R}^{d \times |\mathcal{V}|}$ matrix, each column is node embedding [what we learn!]

$\mathbf{v} \in \mathbb{I}^{|\mathcal{V}|}$ indicator vector, all zeroes except a one in column indicating node v

“Shallow” Encoding Cont.

- Simplest encoding approach: **encoder is just an embedding-lookup**



“Shallow” Encoding

- Simplest encoding approach: **encoder is just an embedding-lookup**
i.e., each node is assigned a unique embedding vector.
- E.g., node2vec, DeepWalk, LINE
- **We will focus on shallow encoding in this class ...**
- Next class we will discuss more encoders based on **deep neural networks**

How to Define Node Similarity?

- Key distinction between “shallow” methods is **how they define node similarity**
- E.g., should two nodes have similar embeddings if they....
 - Are connected?
 - Share neighbors?
 - In the same community?
 - Have similar “structural roles”?
 - ...?

Different Ways of Node Similarity

1. Adjacency-based similarity

2. Multi-hop similarity

3. Random walk approaches

High-level structure and material from:

- Hamilton et al. 2017. Representation Learning on Graphs: Methods and Applications. *IEEE Data Engineering Bulletin on Graph Systems*.

Adjacency-based Similarity

Ahmed et al. 2013. [Distributed Natural Large Scale Graph Factorization](#). *WWW*.

Adjacency-based Similarity

- **Similarity function** is just the edge weight between u and v in the original network
- **Intuition:** Dot products between node embeddings approximate edge existence

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \| \mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v} \|^2$$

loss (what we want to minimize)

sum over all node pairs

embedding similarity

(weighted) adjacency matrix for the graph

Adjacency-based Similarity

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}\|^2$$

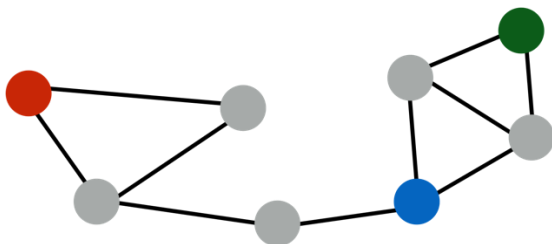
- Find embedding matrix $\mathbf{Z} \in \mathbb{R}^{d \times |V|}$ that minimizes the loss $\mathcal{L}$
 - Option 1: Use stochastic gradient descent (SGD) as a general optimization method.
 - Highly scalable, general approach
 - Option 2: Solve matrix decomposition solvers (e.g., SVD or QR decomposition routines).
 - Only works in limited cases.

Adjacency-based Similarity

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}\|^2$$

- **Drawbacks:**

- $O(|V|^2)$ runtime (Must consider all node pairs)
 - Can make $O(|E|)$ by only summing over non-zero edges and using regularization (e.g., [Ahmed et al., 2013](#))
- $O(|V|)$ parameters! (One learned vector per node).
- Only considers direct, local connections.



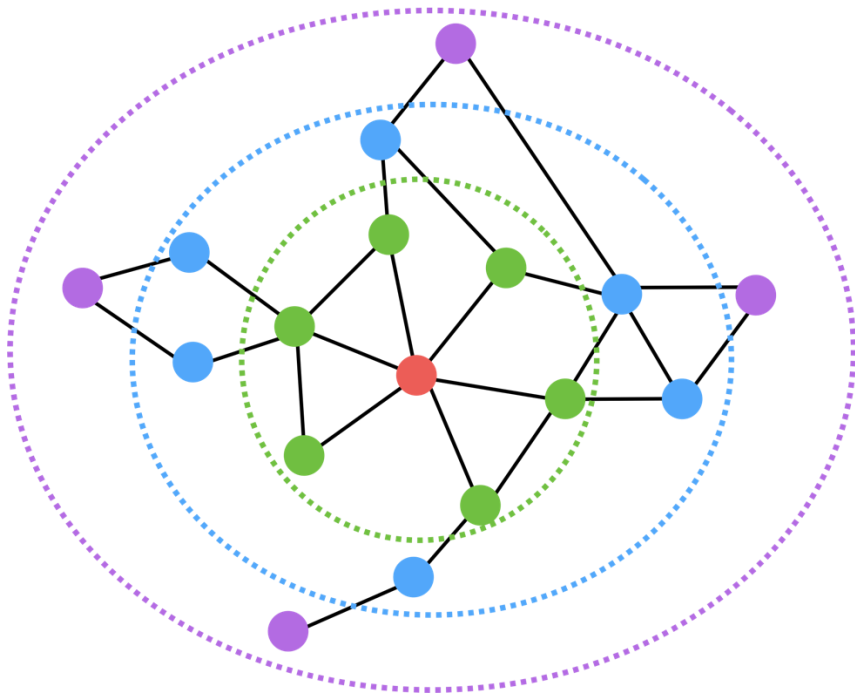
e.g., the **blue** node is obviously more similar to **green** compared to **red** node, despite none having direct connections.

Multi-hop Similarity

- Cao et al. 2015. [GraRep: Learning Graph Representations with Global Structural Information](#). *CIKM*.
- Ou et al. 2016. [Asymmetric Transitivity Preserving Graph Embedding](#). *KDD*.

Multi-hop Similarity

- **Idea:** Consider k-hop node neighbors.
 - E.g., two or three-hop neighbors.



- **Red:** Target node
- **Green:** 1-hop neighbors
 - A (i.e., adjacency matrix)
- **Blue:** 2-hop neighbors
 - A^2
- **Purple:** 3-hop neighbors
 - A^3

Multi-hop Similarity

- **Basic idea:** $\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}^k\|^2$
- Train embeddings to predict k-hop neighbors.
- In practice ([GraRep from Cao et al, 2015](#)):
 - Use log-transformed, probabilistic adjacency matrix:

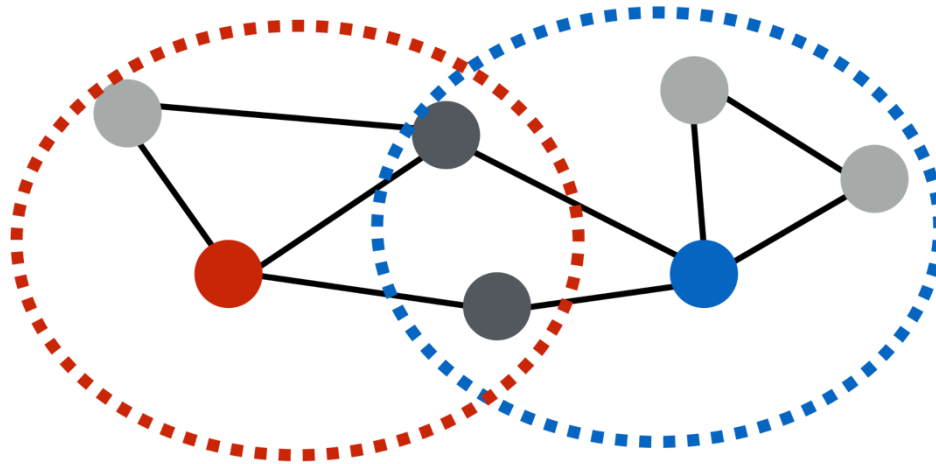
$$\tilde{\mathbf{A}}_{i,j}^k = \max \left(\log \left(\frac{(\mathbf{A}_{i,j}/d_i)}{\sum_{l \in V} (\mathbf{A}_{l,j}/d_l)^k} \right)^k - \alpha, 0 \right)$$

node degree constant shift

- Train multiple different hop lengths and concatenate output.

Multi-hop Similarity

- **Another option: Measure overlap between node neighborhoods.**



- Example overlap functions:
 - Jaccard similarity
 - Adamic-Adar score

Multi-hop Similarity

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \left\| \boxed{\mathbf{z}_u^\top \mathbf{z}_v} - \boxed{\mathbf{S}_{u,v}} \right\|^2$$

embedding similarity

multi-hop network similarity
(i.e., any neighborhood overlap measure)

- $\mathbf{S}_{u,v}$ is the neighborhood overlap between u and v (e.g., Jaccard overlap or Adamic-Adar score).
- This technique is known as [HOPE \(Yan et al., 2016\)](#).

Summary So Far

- **Basic idea so far:**
 - 1) Define pairwise node similarities.
 - 2) Optimize low-dimensional embeddings to approximate these pairwise similarities.
- **Issues:**
 - **Expensive:** Generally $O(|V|^2)$, since we need to iterate over all pairs of nodes.
 - **Brittle:** Must hand-design deterministic node similarity measures.
 - **Massive parameter space:** $O(|V|)$ parameters

Random Walk Approaches

- Perozzi et al. 2014. [DeepWalk: Online Learning of Social Representations](#). *KDD*.
- Grover et al. 2016. [node2vec: Scalable Feature Learning for Networks](#). *KDD*.

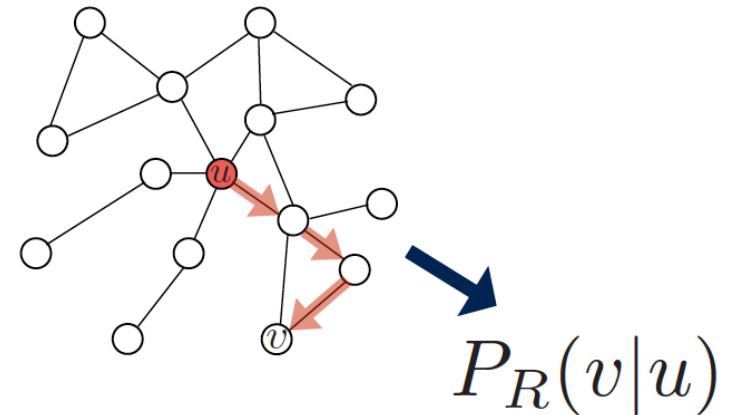
Random-walk Embeddings

$$\mathbf{z}_u^\top \mathbf{z}_v \approx$$

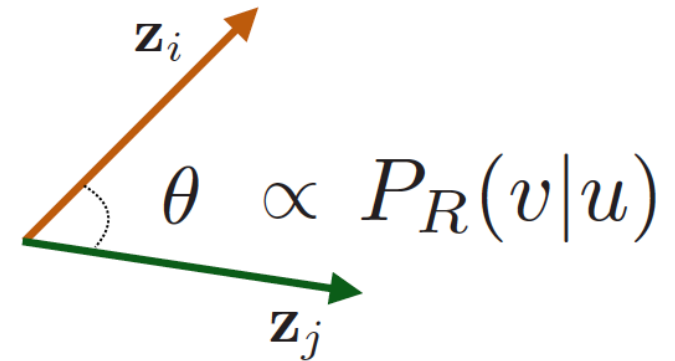
probability that u and v co-occur on a random walk over the network

Random-walk Embeddings

1. Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R .



2. Optimize embeddings to encode these random walk statistics.



Why Random Walks?

- 1. Expressivity:** Flexible stochastic definition of node similarity that incorporates both local and higher-order neighborhood information
- 2. Efficiency:** Do not need to consider all node pairs when training; only need to consider pairs that co-occur on random walks

Random Walk Optimization

1. Run short random walks starting from each node on the graph using some strategy R
2. For each node u collect $N_R(u)$, the multiset of nodes visited on random walks starting from u
3. Optimize embeddings to according to:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

* $N_R(u)$ can have repeat elements since nodes can be visited multiple times on random walks.

Random Walk Optimization

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v | \mathbf{z}_u))$$

- **Intuition:** Optimize embeddings to maximize likelihood of random walk co-occurrences.
- **Parameterize** $P(v | \mathbf{z}_u)$ **using softmax:**

$$P(v | \mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)}$$



Questions?